POSTMAN HANDBOOK

1. Postman Fundamentals & API Workflow.

What is Postman ?

Postman is an API platform used to build, test, document and share APIs. It helps developers and testers to send requests, get responses and automate API workflows easily.

Why we use Postman ?

- Test APIs quickly
- Automate API testing
- Manage environments & variables
- Organize APIs in collections
- Share & collaborate with teams
- Debug and validate responses
- Simulate real-world API workflows

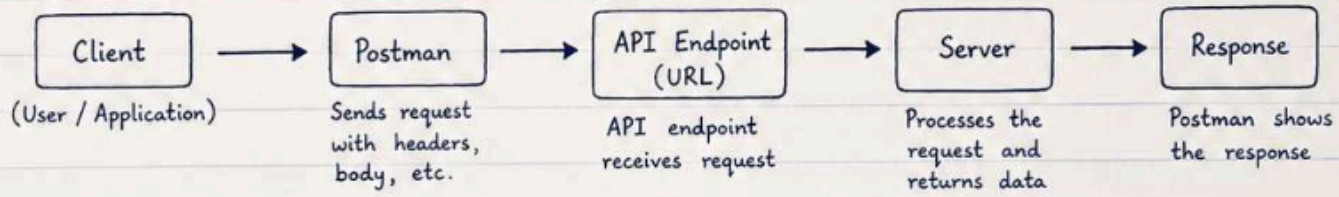
HTTP Methods

GET	– Retrieve data from the server.
POST	– Send data to the server to create a new resource.
PUT	– Update the entire resource.
PATCH	– Update part of the resource.
DELETE	– Delete a resource.

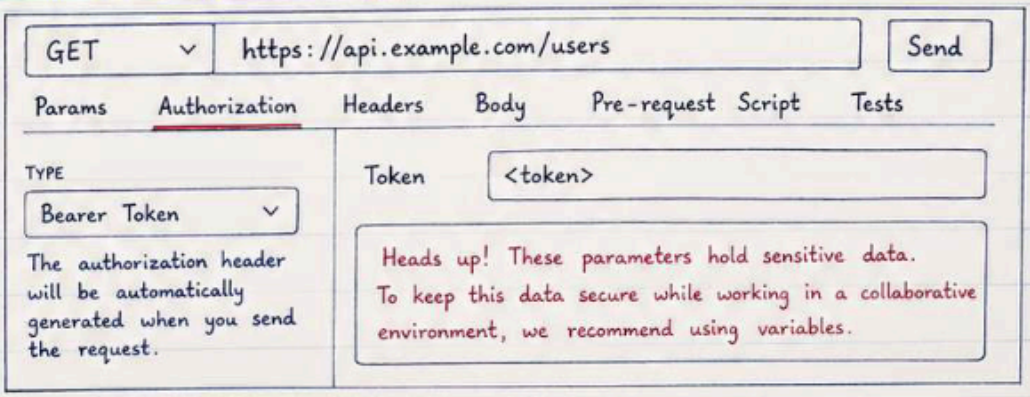
Key Components of an API Request

URL / Endpoint	– The API address to which request is sent.
Query Parameters	– Key-value pairs in the URL.
Headers	– Metadata sent with the request.
Request Body	– Data sent to the server (mainly in POST, PUT, PATCH).
Authorization	– Used to authenticate the request.
Response	– Data returned by the server.
Status Codes	– Indicate the result of the request.

API Workflow (Request → Server → Response)



Basic API Request Example



Common Status Codes

200	– OK (Success)
201	– Created
400	– Bad Request
401	– Unauthorized
403	– Forbidden
404	– Not Found
500	– Internal Server Error

Important

Postman is not the API. It is a tool that helps you communicate with the API. ★

POSTMAN HANDBOOK

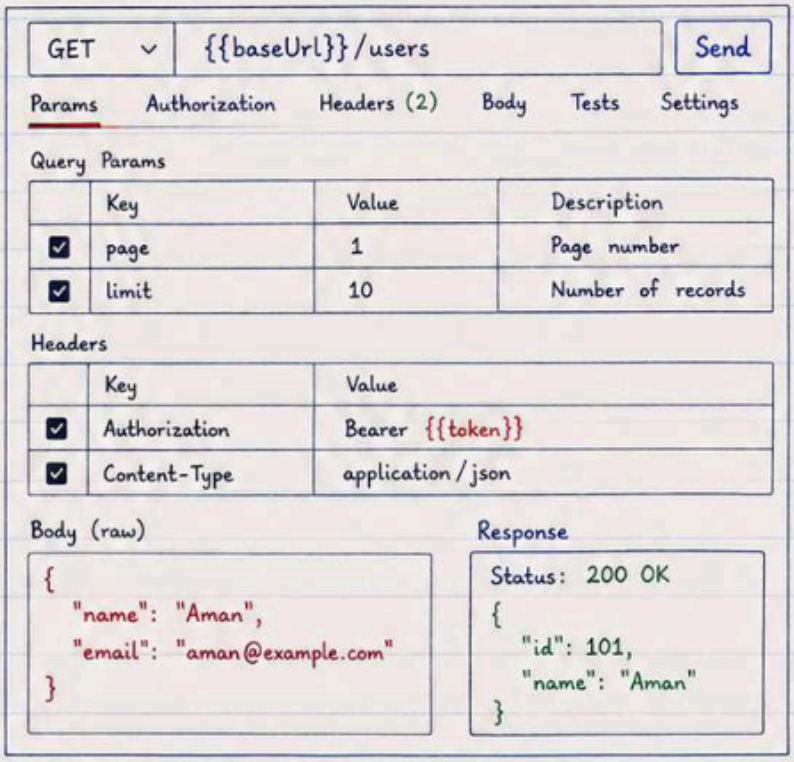
2. Requests, Collections & Environments

★ REQUESTS

A request is an API call made to a specific endpoint.

How to create a request :

① Select HTTP Method (GET, POST, etc.)
② Enter the URL / Endpoint
③ Add Query Parameters (Params)
④ Add Headers
⑤ Add Body (if required)
⑥ Add Authorization (if required)
⑦ Click Send → Get Response



★ COLLECTIONS

Collections help you organize and group related APIs.

```
Collection: My Project APIs
├── Authentication
│   ├── Login
│   └── Logout
├── Users
│   ├── Create User
│   ├── Get User
│   └── Update User
├── Orders
│   ├── Create Order
│   └── Get Order
├── Payments
│   ├── Initiate Payment
│   └── Get Payment Status
└── Reports
    └── Generate Report
```

What you can do with Collections :

- Create a new Collection
- Add requests inside the Collection
- Create folders inside Collection
- Drag & drop to reorder requests
- Duplicate / Clone requests
- Share collections with team
- Export / Import collections

★ ENVIRONMENTS

Environments allow you to store values (variables) and use them in your requests.
This helps you easily switch between different setups.

Example Environments :

- Development
- QA
- Staging
- Production

Why use Environments ?

- Different base URLs
- Different tokens / credentials
- Easy to switch without changing requests
- Keep secrets safe

Example Variables in an Environment

Variable	Current Value
baseUrl	https://qa.example.com
token	eyJhbGciOiJIUzI1NiJ9...
userId	101

★ Well organized collections + environments = clean, maintainable & reusable APIs.

POSTMAN HANDBOOK

3. VARIABLES : The Most Important Postman Concept

Variables help you store and reuse values in your requests, scripts and environments.

Types of Variables

① Global Variables
② Collection Variables
③ Environment Variables
④ Local Variables
⑤ Data Variables

Variable Syntax

Use double curly braces to access variables.

{{baseUrl}}
{{token}}
{{userId}}

Example

Instead of hardcoding values:

https://qa.example.com/users/123 ✗

Use variables:

{{baseUrl}}/users/{{userId}} ✓

Example Values

baseUrl → https://qa.example.com
userId → 123
token → abc123xyz
apiKey → 5f6g7h8i9

Variable Scope (Resolution Order)

Global ↓ Collection ↓ Environment ↓ Local ↓ Data

↑ Highest Priority
↓ Lowest Priority

Postman searches for a variable in the above order. If found at a higher level, it will override the lower level variable.

Where to Use Variables ?

- Request URL
- Query Parameters
- Headers
- Request Body
- Authorization
- Pre-request Scripts
- Test Scripts
- Anywhere in Postman

Initial Value vs Current Value

Initial Value	Current / Local Value
• Default value saved in variable • Loaded every time environment is selected • Used if current value is empty	• Temporary value used during session • Can be changed by scripts • Overrides initial value

Dynamic Variables (Built-in by Postman)

{{timestamp}} → Current timestamp (milliseconds)
{{randomInt}} → Random integer
{{guid}} → Generate GUID
{{baseUrl}} → Custom environment variable

Why Variables ?

✓ Reusability
✓ Easy maintenance
✓ Environment switching
✓ Secure storage of sensitive data
✓ Dynamic & flexible API testing

Practical Example

GET {{baseUrl}}/users/{{userId}}
Headers :
Authorization: Bearer {{token}}
Content-Type: application/json

→

Environment Values (QA)
baseUrl = https://qa.example.com
userId = 123
token = abc123xyz

→

Final Request Sent
GET https://qa.example.com/users/123
Authorication: Bearer abc123xyz
Content-Type: application/json

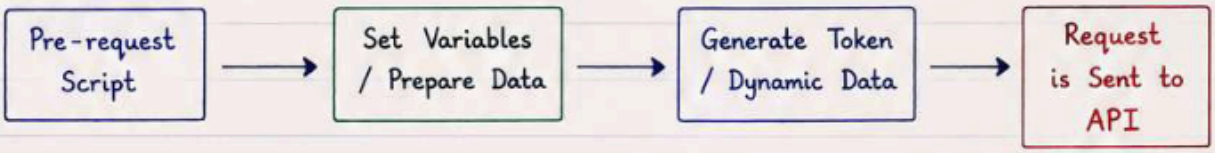
★ Mastering variables is the key to building scalable and maintainable API workflows.

POSTMAN HANDBOOK

4. PRE-REQUEST SCRIPTS

Pre-request scripts are JavaScript code that runs **BEFORE** the request is sent.

WHERE IT RUNS ?



COMMON USE CASES

- Generate dynamic data (email, name, id)
- Set timestamps
- Generate random values
- Create authentication data / tokens
- Set request variables
- Prepare request payload

EXAMPLE 1: Generate Dynamic Email

```
// Generate unique email using timestamp
const email = `user_${Date.now()}@test.com`;
pm.variables.set("email", email);
console.log("Generated Email:", email);
```

Use in request body:

```
{
  "email": "{{email}}"
}
```

EXAMPLE 2: Set Timestamp

```
const timestamp = Date.now();
pm.variables.set("timestamp", timestamp);
console.log("Timestamp:", timestamp);
```

Use in request:

```
Order created at: {{timestamp}}
```

EXAMPLE 3: Generate Random Integer

```
const randomInt = Math.floor(Math.random() * 1000);
pm.variables.set("randomInt", randomInt);
console.log("Random Integer:", randomInt);
```

Use in request:

```
"referenceId": "{{randomInt}}"
```

EXAMPLE 4: Prepare Auth Data (Basic Auth)

```
const username = pm.environment.get("username");
const password = pm.environment.get("password");
const auth = Buffer.from(username + ":" + password).
             toString('base64');
pm.variables.set("authHeader", `Basic ${auth}`);
```

Use in Headers:

Key	Value
Authorization	{{authHeader}}

EXAMPLE 5: Generate OAuth Token

```
pm.sendRequest({
   url: "{{tokenUrl}}",
   method: 'POST',
   header: { 'Content-Type': 'application/json' },
   body: { mode: 'raw', raw: JSON.stringify({
         client_id: '{{clientId}}',
         client_secret: '{{clientSecret}}',
         grant_type: 'client_credentials'
    })}
}, function (err, res) {
    const token = res.json().access_token;
    pm.environment.set("token", token);
});
```

How to Add Pre-request Script ?

① Open a request
② Go to "Pre-request Script" tab
③ Write your JavaScript code
④ Save and send request

IMPORTANT POINTS	
	★ Pre-request script runs every time before the request. ★ Great for dynamic data, auth, and setting variables. ★ Improves reusability and automation.

POSTMAN HANDBOOK

5. POST-RESPONSE / TEST SCRIPTS

Test scripts are JavaScript code that runs AFTER the response is received.

HOW IT WORKS ?



COMMON VALIDATIONS

- Status code validation
- Response body validation
- Headers validation
- JSON parsing
- Assertions
- Multiple tests
- Set variables from response
- Console debugging

EXAMPLE 1: Status Code Validation

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
```

EXAMPLE 2: Validate Response Body

```
pm.test("Response has userId", function () {
    const json = pm.response.json();
    pm.expect(json.userId).to.exist;
    pm.expect(json.name).to.be.a('string');
});
```

EXAMPLE 3: Validate Headers

```
pm.test("Content-Type is application/json", function () {
    pm.response.to.have.header("Content-Type");
    pm.expect(pm.response.headers.get("Content-Type"))
        .to.include("application/json");
});
```

EXAMPLE 4: Extract Data & Set Variable

```
const json = pm.response.json();
pm.environment.set("userId", json.id);
pm.collectionVariables.set("userName", json.name);
```

MULTIPLE TESTS EXAMPLE

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
pm.test("Response time is less than 500ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(500);
});
pm.test("Response has success field", function () {
    const json = pm.response.json();
    pm.expect(json.success).to.be.true;
});
```

DEBUGGING IN TEST SCRIPTS

- console.log() → Print values in Postman Console
- Useful when scripts fail

```
const json = pm.response.json();
console.log("Full Response:", json);
console.log("User ID:", json.id);
console.log("Headers:", pm.response.headers.all());
```

Note :
→ Test scripts do NOT modify the response.
→ They only validate and extract data.
→ They help in automated validation of APIs.

ASSERTIONS (pm.expect)

Assertion	Example	Assertion	Example
Exist	pm.expect(json.id).to.exist;	True	pm.expect(json.success).to.be.true;
Equal	pm.expect(json.status).to.eql("OK");	Above	pm.expect(json.count).to.be.above(0);
Include	pm.expect(json.message).to.include("success");	Below	pm.expect(json.time).to.be.below(500);
Length	pm.expect(json.items).to.have.length(3);	Type	pm.expect(json.name).to.be.a('string');

PRO TIP : Use tests to validate every important field in the response.

POSTMAN HANDBOOK

6. API CHAINING & PASSING DATA BETWEEN REQUESTS

In real applications, APIs are not called in isolation.

We extract data from one response and use it in the next request.

EXAMPLE WORKFLOW

1. Login (Get Token) → 2. Create User → 3. Get User (Using userId) → 4. Update User → 5. Delete User

STEP 1: Login API → Get Token

Request:

```
POST /api/login
```

Response:

```
{
  "token": "abc123xyz",
  "expiresIn": 3600
}
```

Test Script → Extract Token

```
const response = pm.response.json();
pm.environment.set("token", response.token);

// token is now saved in environment variable
```

Use in next request:

- Authorization
  Type: Bearer Token
  Token: {{token}}

STEP 2: Create User API → Extract userId

Request:

```
POST /api/users
Body (raw - JSON):
{
  "name": "Aman",
  "email": "user@test.com"
}
```

Response:

```
{
  "id": 101,
  "name": "Aman",
  "email": "user@test.com"
}
```

Test Script → Extract userId

```
const res = pm.response.json();
pm.environment.set("userId", res.id);

// userId is saved for next APIs
```

STEP 3: Use userId in Next API

Request:

```
GET /api/users/{{userId}}
```

Authorization Header:

Authorization: Bearer {{token}}

Path Parameter:

userId → {{userId}}

STEP 4: Update User API

Request:

```
PUT /api/users/{{userId}}
Body (raw - JSON):
{
  "name": "Aman Updated",
  "email": "aman.updated@test.com"
}
```

Response:

```
{
  "id": 101,
  "name": "Aman Updated",
  "email": "aman.updated@test.com"
}
```

STEP 5: Delete User API

Request:

```
DELETE /api/users/{{userId}}
```

Response:

```
{
  "message": "User deleted successfully"
}
```

KEY TAKEAWAYS

- Extract data from response
- Store in variables
- Use variables in next requests
- Chain multiple APIs
- Build dynamic workflows

VARIABLE TYPES USED IN CHAINING

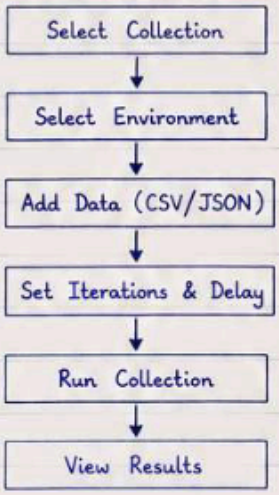
Environment Variables > Collection Variables > Local Variables (pm.variables.set)

POSTMAN HANDBOOK

7. COLLECTION RUNNER & DATA-DRIVEN TESTING

Collection Runner allows you to run a collection of requests automatically with different data (CSV/JSON).

COLLECTION RUNNER FLOW



HOW TO RUN A COLLECTION

① Click on the collection
② Click on ▶ Run (Collection Runner)
③ Select Environment
④ Select Data File (if any)
⑤ Set Iterations, Delay, Request Order
⑥ Click Run Collection
⑦ View Results (Passed / Failed)

COLLECTION RUNNER SETTINGS

- Environment : Choose the environment (QA/Staging/Prod)
- Data : Add CSV or JSON file
- Iterations : How many times to run
- Delay : Delay between each request (ms)
- Data File Type : CSV / JSON
- Stop on Error : Stop the run when a request fails
- Request Order : Run in sequence

DATA-DRIVEN TESTING

We can run the same API multiple times with different data.

EXAMPLE CSV FILE (users.csv)

username	password	email
user1	test123	user1@test.com
user2	test456	user2@test.com
user3	test789	user3@test.com

USE IN REQUEST

POST /api/login

Body (raw - JSON):

```
{
  "username": "{{username}}",
  "password": "{{password}}"
}
```

IN PRE-REQUEST SCRIPT

```
pm.variables.set("username",
  pm.iterationData.get("username"));
pm.variables.set("password",
  pm.iterationData.get("password"));
```

DATA VARIABLES

Ways to Access Data	Example
• Using {{variableName}}	• {{email}}
• Using pm.iterationData.get("columnName")	• pm.iterationData.get("email")

TIPS

★ Keep CSV column names same as variables.
★ First row of CSV should be column names.
★ Use environment variables for base URL, tokens, common data.

SUPPORTED DATA FILE FORMATS

- CSV (Comma Separated Values)
- JSON (Array of JSON objects)

EXAMPLE JSON FILE (users.json)

```
[
  { "username": "user1", "password": "test123", "email": "user1@test.com" },
  { "username": "user2", "password": "test456", "email": "user2@test.com" },
  { "username": "user3", "password": "test789", "email": "user3@test.com" }
]
```

RESULTS & REPORTS

- After the run, Postman shows summary (Total, Passed, Failed, Skipped)
- You can click on each request to see details
- Export results (JSON) if needed
- Failed tests help you identify issues quickly

WHAT YOU CAN AUTOMATE

✓ Functional testing
✓ Regression testing
✓ Smoke testing
✓ Data validation
✓ Multiple scenarios

POSTMAN HANDBOOK

8. ADVANCED AUTOMATION & REAL-WORLD WORKFLOW

Postman is not just a tool to send requests. With scripts, variables and collections, we can build complete automated API workflows.

COMPLETE AUTOMATED WORKFLOW

1. Set Environment
2. Authenticate (Login)
3. Get Token
4. Create Data
5. Extract ID / Data
6. Use Data in Next API
7. Validate Response (Tests)
8. Run Collection
9. Generate Report

COLLECTION LEVEL SCRIPTS

- Collection level pre-request script runs before every request in the collection,
- Useful for common setup like setting base URL, tokens etc.

Example (Collection Pre-request Script)

```
pm.environment.set("baseUrl", "https://qa.api.com");
pm.collectionVariables.set("appVersion", "v1.0.0");
```

REQUEST CHAINING EXAMPLE (CRUD FLOW)

Login → Create → Get → Update → Delete

USEFUL POSTMAN METHODS

- pm.environment.set(key, value) : Set environment variable
- pm.collectionVariables.set(key, value) : Set collection variable
- pm.variables.set(key, value) : Set local variable
- pm.response.json() : Parse JSON response
- pm.expect(value).to.eql(expected) : Assertion
- pm.test("name", function() {}) : Write a test
- pm.iterationData.get("key") : Get data from CSV/JSON
- console.log("message") : Debug in Postman Console

AUTOMATION WITH NEWMAN

- Newman is the command line collection runner for Postman.
- It allows you to run collections from terminal / CI tools.
- Great for automation and CI/CD pipelines.

Example:

```
newman run collection.json -e environment.json -d data.csv -r cli,json
```

NEWMAN REPORTERS

Reporter	Description
cli	Shows result in terminal
json	Generates JSON report
html	Generates HTML report
junit	Generates JUnit XML report

INTEGRATION WITH CI/CD PIPELINE

1. Code Push (Git) → 2. CI/CD Pipeline Triggered → 3. Install Newman & Dependencies → 4. Run Postman Collection (Newman) → 5. Generate Test Report → 6. Publish Report & Notify

Example: Jenkins Pipeline (Snippet)

```
pipeline {
  agent any
  stages {
    stage('Run API Tests') {
      steps {
        sh 'npm install -g newman'
        sh 'newman run collection.json -e environment.json -r cli,html --reporter-html-export report.html'
      }
    }
  }
  post {
    always {
      publishHTML(target: [ allowMissing: false, alwaysLinkToLastBuild: true,
          keepAll: true, reportDir: '.', reportFiles: 'report.html', reportName: 'Postman Report' ])
    }
  }
}
```

BEST PRACTICES

- ★ Use environments for different stages.
- ★ Store tokens in environments, not in requests.
- ★ Use variables to avoid hardcoding.
- ★ Write tests for every important API.
- ★ Keep collections organized.
- ★ Run collections in CI/CD for reliability.